

MANUAL DE USUARIO

Sistema de Gestión Académica y Planificación de Rutas de Aprendizaje

Curso	Estructuras de Datos
Programa	Ingeniería de Sistemas
Docente	Ing. Jhon Haide Cano Beltran.
Estudiante	Steven Zuñiga Monsalve
Fecha	Mayo 2026

1. Descripción General

El Sistema de Gestión Académica es una aplicación de consola desarrollada en Java que permite administrar estudiantes, materias, horarios, rutas entre edificios y operaciones con soporte de deshacer/rehacer. El sistema implementa las principales estructuras de datos: HashMap, TreeMap, LinkedList, Stack, Queue y matrices nativas.

Requisitos del sistema:

Componente	Requisito
Java	JDK 17 o superior
IDE	VS Code con Extension Pack for Java
Sistema Operativo	Windows, Linux o macOS
Memoria RAM	512 MB mínimo

2. Estructura del Proyecto

El proyecto está organizado en los siguientes paquetes:

```
Trabajo-Final-Estructura-De-Datos/ src/ Excepciones/ -> Clases de excepcion personalizadas
Modelo/ -> Persona, Estudiante, Materia, Aula, Facultad
Servicios/ -> Logica de negocio (gestores)
Util/ -> Dijkstra, ProcesadorBatch
Main.java -> Clase principal con menu interactivo
inscripciones.csv -> Archivo de prueba para procesamiento batch
```

3. Cómo Ejecutar el Sistema

Desde VS Code:

1. Clonar o abrir el repositorio en VS Code.
2. Abrir el archivo Main.java.
3. Hacer click en el botón Run que aparece encima del método main.
4. El menú principal aparecerá en la terminal integrada.

Desde terminal:

```
javac -cp src src/Excepciones/*.java src/Modelo/*.java src/Servicios/*.java  
src/Util/*.java Main.java java -cp src:. Main
```

4. Menú Principal

Al ejecutar el sistema verá el siguiente menú:

```
===== PLANIFICACION  
ACADEMICA - SISTEMA UNIVERSITARIO  
=====  
GESTION DE  
ESTUDIANTES === 1. Registrar estudiante 2. Buscar estudiante por ID 3. Listar  
todos los estudiantes 4. Eliminar estudiante === GESTION DE MATERIAS === 5.  
Crear materia 6. Agregar pre-requisito 7. Mostrar pre-requisitos 8. Inscribir  
estudiante en materia 9. Cancelar inscripcion 10. Mostrar cola de espera ===  
GESTION DE HORARIOS === 11. Registrar aula 12. Reservar horario 13. Liberar  
horario 14. Consultar disponibilidad === RUTAS ENTRE EDIFICIOS === 15. Mostrar  
edificios 16. Calcular ruta mas corta === REPORTE ACADEMICOS === 17.  
Registrar nota 18. Ver reporte academico 19. Volver al reporte anterior ===  
DESHACER / REHACER === 20. Deshacer ultima operacion 21. Rehacer ultima  
operacion === PROCESAMIENTO POR LOTES === 22. Procesar archivo CSV ===  
FACULTADES === 23. Listar facultades === SALIR === 0. Salir
```

5. Funcionalidades del Sistema

5.1 Gestión de Estudiantes

Opción	Descripción	Datos requeridos
1	Registrar estudiante	ID, Nombre, Email, Semestre
2	Buscar por ID	ID del estudiante
3	Listar todos	Ninguno
4	Eliminar estudiante	ID del estudiante

Ejemplo de registro:

```
Seleccione una opcion: 1 ID: 2024001 Nombre: Ana Maria Gomez Email:  
ana@universidad.edu Semestre actual: 3 Estudiante registrado exitosamente.
```

5.2 Gestión de Materias

Opción	Descripción	Datos requeridos
5	Crear materia	Código, Nombre, Cupos, Créditos
6	Agregar pre-requisito	Código materia, Código pre-req
7	Mostrar pre-requisitos	Código de la materia
8	Inscribir estudiante	ID estudiante, Código materia
9	Cancelar inscripción	ID estudiante, Código materia
10	Ver cola de espera	Código de la materia

Nota: Si una materia tiene cupos llenos, el estudiante se agrega automáticamente a la cola de espera. Cuando se cancela una inscripción, el primer estudiante en la cola toma el cupo liberado.

5.3 Gestión de Horarios

Cada aula maneja una matriz boolean[7][24] donde las filas representan los días de la semana y las columnas las horas del día (0-23).

```
Días: 0=Domingo 1=Lunes 2=Martes 3=Miercoles 4=Jueves 5=Viernes 6=Sabado
Ejemplo de reserva: Nombre del aula: 101 Dia (1=Lunes): 1 Hora inicio (0-23): 8
Duracion en horas: 2 -> Lunes 8:00 RESERVADO -> Lunes 9:00 RESERVADO
```

5.4 Rutas entre Edificios (Dijkstra)

El sistema tiene 5 edificios preconfigurados con sus distancias. Usa el algoritmo de Dijkstra para encontrar la ruta más corta.

```
Edificios disponibles: 0: Ingenieria 1: Biblioteca 2: Cafeteria 3: Rectoria 4: Laboratorios
Ejemplo: Indice edificio origen: 0 Indice edificio destino: 3 -> Ruta: Ingenieria -> Biblioteca -> Rectoria -> Distancia total: 220 metros
```

5.5 Reportes Académicos

Las notas se almacenan en un arreglo Double[10][20] por estudiante (10 semestres x 20 materias). El reporte calcula promedios y detecta materias reprobadas (nota menor a 3.0).

```
Para registrar nota: ID del estudiante: 2024001 Semestre (1-10): 1 Indice materia (1-20): 1 Nota (0.0 - 5.0): 4.5
```

5.6 Sistema Deshacer / Rehacer

Operaciones que pueden deshacerse:

- Registrar estudiante
- Eliminar estudiante
- Inscribir estudiante en materia
- Cancelar inscripción
- Reservar horario en aula

```
Opcion 20 = Deshacer ultima operacion Opcion 21 = Rehacer ultima operacion deshecha
```

5.7 Procesamiento por Lotes (Batch)

Permite procesar inscripciones masivas desde un archivo CSV. El formato de cada línea debe ser: `idEstudiante,codigoMateria`

```
Formato del archivo inscripciones.csv: 2024001,CALC101 2024002,CALC101
2024003,FIS101 Seleccione opcion 22 e ingrese la ruta del archivo. El sistema
procesara cada solicitud en orden y mostrara un resumen.
```

6. Manejo de Errores

El sistema maneja los siguientes errores de forma controlada:

Excepción	Cuándo ocurre
EstudianteNoEncontradoException	Se busca un ID que no existe
PreRequisitoNoAprobadoException	El estudiante no cumple pre-requisitos
CupoLlenoException	La materia no tiene cupos disponibles
HorarioConflictivoException	El horario solicitado ya está ocupado
PilaDeshacerVacíaException	No hay operaciones para deshacer/rehacer
ColaDeEsperaVacíaException	Se consulta una cola de espera vacía
ArchivoInvalidoException	El archivo CSV no existe o tiene formato incorrecto

7. Estructuras de Datos Implementadas

Estructura	Uso en el sistema
HashMap<String, Estudiante>	Busqueda rapida de estudiantes por ID O(1)
TreeMap<String, Aula>	Aulas ordenadas alfabeticamente
LinkedList<String>	Pre-requisitos e historial de materias
ArrayDeque (Queue)	Cola de espera y procesamiento batch
Stack<Object[]>	Deshacer/rehacer y navegacion de reportes
boolean[7][24]	Disponibilidad horaria de cada aula
int[N][N]	Matriz de distancias entre edificios
Double[10][20]	Notas por semestre de cada estudiante
Facultad[5]	Lista fija de facultades de la universidad

8. Flujo Típico de Uso

Secuencia recomendada para usar el sistema:

- Paso 1:** Registrar facultades (opcion 23) — ya estan precargadas.
- Paso 2:** Registrar aulas (opcion 11) con nombre y capacidad.
- Paso 3:** Registrar estudiantes (opcion 1) con sus datos.

Paso 4: Crear materias (opcion 5) con cupos y creditos.

Paso 5: Agregar pre-requisitos a las materias (opcion 6) si aplica.

Paso 6: Reservar horarios en aulas (opcion 12).

Paso 7: Inscribir estudiantes en materias (opcion 8).

Paso 8: Registrar notas (opcion 17) y ver reportes (opcion 18).

Paso 9: Usar deshacer/rehacer (opciones 20-21) si es necesario.

Paso 10: Calcular rutas entre edificios (opcion 16).

9. Diagrama de Flujo del Sistema

Flujo general de una inscripcion con cola de espera:

```
[Inicio] | v Seleccionar opcion 8 (Inscribir estudiante) | v Ingresar ID
estudiante y codigo materia | v Verificar pre-requisitos | | [Cumple] [No
cumple] | | v v Hay cupos? PreRequisitoNoAprobadoException | | [Si] [No] | | v
v Inscribir Agregar a cola de espera | | v v Guardar en pilaDeshacer | v [Fin]
```

10. Notas Importantes

- El sistema no persiste datos entre ejecuciones. Al cerrar el programa, todos los datos se pierden.
- El archivo CSV para el batch debe estar en la misma carpeta desde donde se ejecuta el programa.
- Los índices de edificios para Dijkstra van de 0 a 4.
- Las notas van en escala de 0.0 a 5.0. Una nota menor a 3.0 se considera reprobada.
- El sistema Deshacer/Rehacer solo funciona dentro de la misma sesion.